

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Friday 18th September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to identify conceptual structures. The remainder of this chapter is organized as follows:

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, covering the environments, the reinforcement learning setup, and the surrogate policy training.
- Section 1.3 presents the training of the initial and surrogate policies and evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy.
- Section 1.4 provides a quantitative analysis of the latent representations using the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics.
- Section 1.5 provides a qualitative analysis of the resulting clusters by manually examining their semantic content.
- Section 1.6 summarizes the findings and discusses their implications.

1.1 Software Implementation

This section details the experimental evaluation of the proposed method. Deep Learning (DL) and Reinforcement Learning (RL) are strongly empirical fields, where experimental evaluation plays an important role in assessing how methods perform in practice. Particular attention is therefore given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

The CAR implementation is organized as a pipeline of five stages. Each stage performs a specific operation and produces the artifacts required by the subsequent stages. The stages are implemented as independent executables

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space



Figure 1.1: The CAR pipeline.

33 allowing each stage to be rerun independently. The CAR pipeline is illustrated in
 34 Figure 1.1, and Table 1.1 summarizes the libraries used throughout the pipeline.
 35 The remainder of this section describes the purpose and implementation of each
 36 stage.

37 1.1.1 Initial policy training

38 The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018].
 39 RLlib provides scalable implementations of RL algorithms and supports dis-
 40 tributed training. The use of RLlib also allows CAR to be applied to environ-
 41 ments implemented using the Gymnasium interface [Towers et al., 2024]. By
 42 default, the initial policy is trained using Proximal Policy Optimization (PPO)
 43 [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal
 44 Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN)
 45 [Mnih et al., 2013], and DreamerV3 [Hafner et al., 2023], can also be selected
 46 through the same configuration.

47 The policy and critic networks are implemented using PyTorch [Paszke et al., 2019].
 48 PyTorch is the standard deep learning frameworks and is widely used for de-
 49 veloping and training Neural Network (NN). It provides GPU acceleration and
 50 supports a wide range of NN architectures. This makes it well suited for imple-
 51 menting the different policy and critic architectures used in the experiments.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 1.1: Main software libraries used at each stage of the CAR pipeline.

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training NN with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

1.1.4 Surrogate policy evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their latent representations are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

1.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the latent representations. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS, 2023], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

1.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architectures used, and the main parameters of the pipeline.

1.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages:

- **Standard benchmark:** Atari is widely used in the RL community, making the experimental results easier to compare with existing work.
- **Limited environment-specific bias:** The environments were not specifically designed for RL, reducing the risk of introducing biases related to their design.
- **Diversity:** The benchmark provides a diverse set of well-known environments with different types of tasks and levels of complexity, allowing CAR to be evaluated under different conditions.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017]. In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

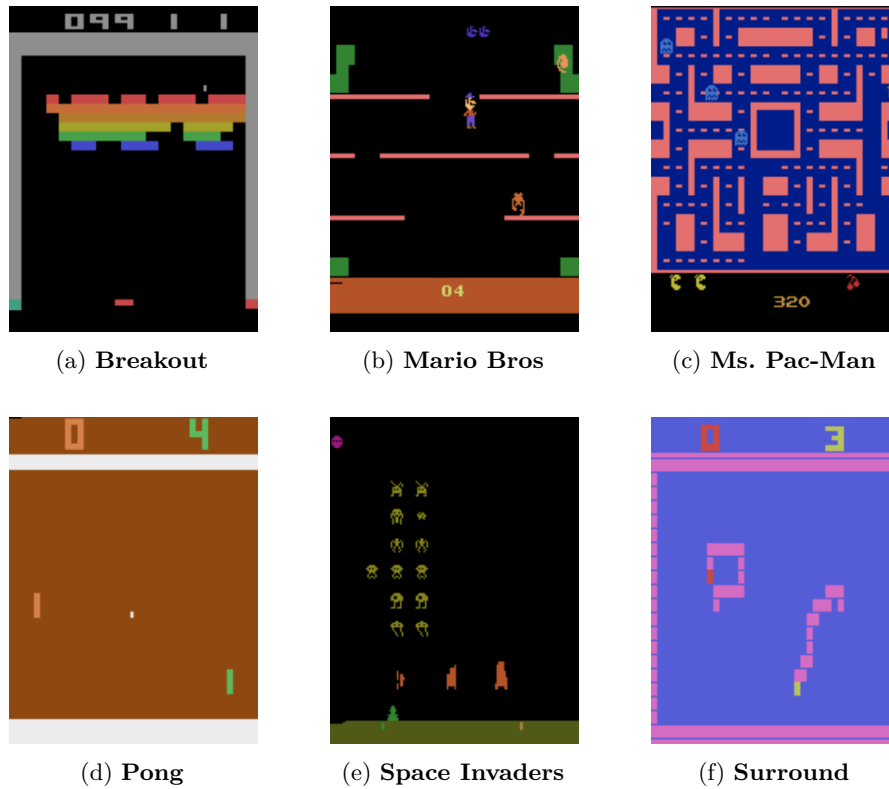


Figure 1.2: The six Atari environments used to evaluate CAR.

To reduce the duration of each episode, frame skipping is used. The agent selects a new action every five frames and repeats it for the following frames. This reduces the number of decisions required during an episode while maintaining sufficient temporal resolution for effective control.

The experiments use the six environments described below::

- **Breakout:** The agent moves a paddle at the bottom of the screen (Figure 1.2a) to bounce a ball into a wall of bricks. Each broken brick scores one point, and the ball speeds up as the wall is cleared.
- **Mario Bros.:** A single-screen platform game (Figure 1.2b) in which the player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block can flip all enemies at once but can only be used a few times per phase.
- **Ms. Pac-Man:** The player moves through a maze (Figure 1.2c), eating pellets while avoiding four ghosts. Power pellets temporarily make the ghosts edible.
- **Pong:** A two-player game (Figure 1.2d) in which the agent controls one paddle and must send the ball past the opponent.
- **Space Invaders:** The agent moves a cannon along the bottom of the screen (Figure 1.2e), firing at descending rows of aliens while avoiding their



Figure 1.3: Architecture of the surrogate policies.

shots and using destructible shields. A bonus saucer occasionally crosses the top of the screen.

- **Surround:** A two-player game on a grid (Figure 1.2f) in which each player leaves a trail behind a moving cursor. The goal is to force the opponent to crash into a trail.

1.2.2 Neural architecture

The surrogate policies use a convolutional NN to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a multilayer perceptron that produces the parameters of the action distribution.

The architecture of the multilayer perceptron varies across environments. For **Pong** and **Space Invaders**, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for **Breakout**, 18 for **Mario Bros**, 9 for **Ms. Pac-Man**, 6 for **Pong**, 6 for **Space Invaders**, and 5 for **Surround**.

Figure 1.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered 1 to 5 for the four convolutional layers and the flattened representation, which are common to all six environments; the representations produced by the multilayer perceptron are then numbered 6a for **Pong** and **Space Invaders** (single hidden layer), or 6b, 7b, and 8b for the other four environments (three hidden layers).

1.2.3 Other CAR parameters

Several other parameters must be defined for the different stages of the CAR pipeline. These parameters are less critical to the overall methodology than those discussed above, but they still need to be specified to reproduce the experiments. Since each stage introduces its own configuration choices, there are too many

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	20 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 20

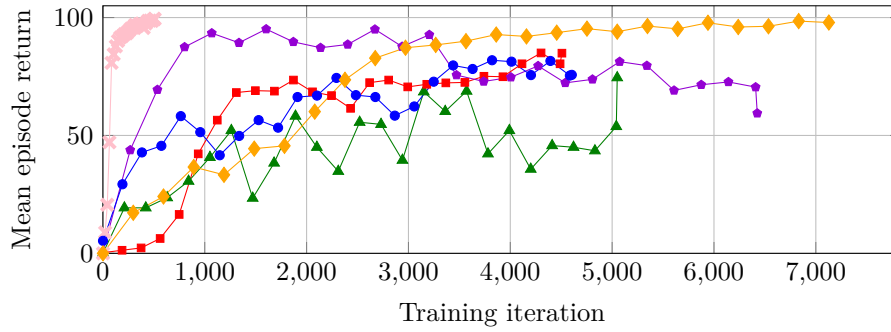
Table 1.2: Main experimental parameters.

parameters to discuss them all in detail. The main parameters used across the experiments are summarized in Table 1.2.

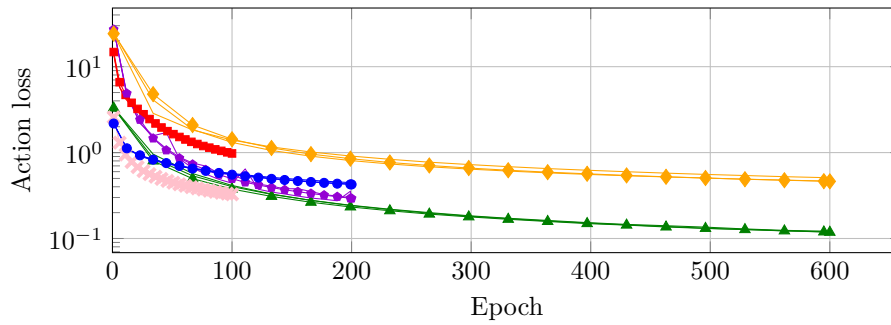
These settings are specific to the experiments presented in this chapter and do not restrict the CAR methodology itself. The different stages of the pipeline are modular and can be configured or replaced depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and assessing their effect on the consistency and meaningfulness of the resulting clusters.

1.3 Policy Training Analysis

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. Section 1.3.1 verifies that the initial policies reach a competent level of play, while Section 1.3.2 evaluates whether the surrogate policies faithfully reproduce their behavior using the *Behavior Reconstruction*.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

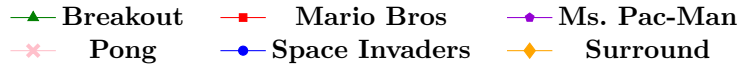


Figure 1.4: Training of the initial and surrogate policies.

1.3.1 Initial policy training

Figure 1.4a shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for **Pong**, to several thousand, for **Surround** and **Ms. Pac-Man**. For the subsequent analysis, the checkpoint with the best performance is retained rather than the final checkpoint.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

1.3.2 Surrogate policy training

Figure 1.4b shows, for each environment, the training action loss of the three surrogate policies. In all cases, the loss decreases smoothly, and the three independently trained policies converge to the same value.

We further evaluate the ability of the surrogate policies to replicate the

Environment	<i>Behavior Reconstruction</i>
Breakout	0.83 $\pm$ 0.03
Mario Bros	0.99 $\pm$ 0.03
Ms. Pac-Man	1.00 $\pm$ 0.03
Pong	0.95 $\pm$ 0.01
Space Invaders	0.97 $\pm$ 0.07
Surround	1.00 $\pm$ 0.01

Table 1.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

behavior of the initial policies using the *Behavior Reconstruction* (Table 1.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for **Breakout** to 1.00 for **Ms. Pac-Man** and **Surround**. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed.

1.4 Quantitative Analysis

This section analyzes the Latent Space (LS) of the surrogate policies using CU, AS, and ACU. The analysis is conducted from two complementary perspectives. First, the evolution of these metrics across the different layers is examined to study the effect of representation depth. Second, for each environment, the layer achieving the highest average ACU is selected and analyzed in more detail across different numbers of clusters.

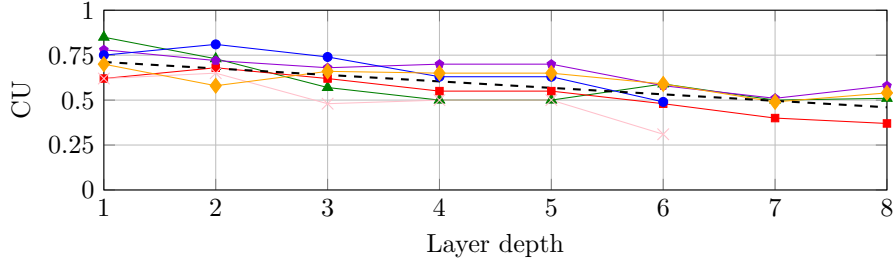
To sample the LS, a random subset of 20,000 observations is selected for each environment. The same subset is then embedded into the LS of each surrogate policy at each analyzed layer. Each point cloud is partitioned using *k*-means clustering [MacQueen, 1967] for each number of clusters from 2 to 20. The three metrics are then computed from these point clouds and their corresponding clusterings. The full results across all environments and analyzed layers are reported in Appendix ??.

1.4.1 Analysis across layers

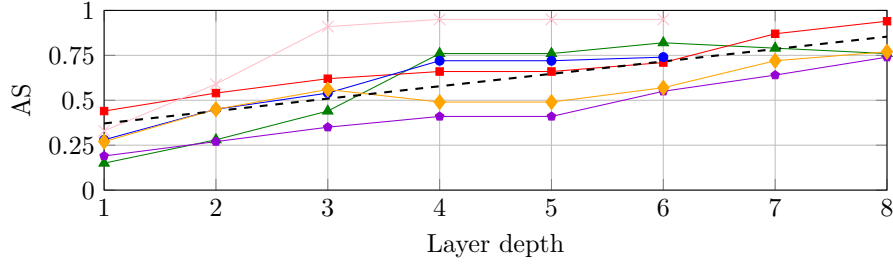
The first analysis examines how the three metrics evolve with the depth of the analyzed layer. For each environment, CU, AS, and ACU are averaged over the tested cluster numbers. Figure 1.5 reports these averages as a function of layer depth.

Across environments, CU tends to be highest in the earliest layers and decreases with depth, while AS follows the opposite trend and increases as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization. As ACU combines these two effects, its evolution depends on the environment.

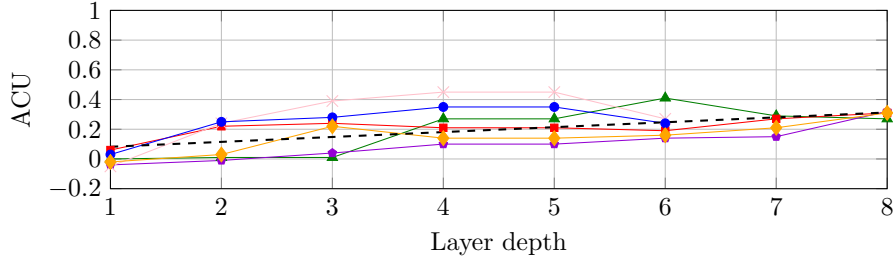
The corresponding linear fits show that CU decreases from 0.72 at the first layer to 0.50 at the last, with a slope of approximately -0.04 per layer, while AS increases from 0.28 to 0.80, with a slope of approximately $+0.07$ per layer. Over the same range, ACU increases more moderately from 0.00 to 0.30, with a slope



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.



Figure 1.5: CU, AS, and ACU averaged over the number of clusters $K \in [2, 20]$, with respect to the depth of the analyzed layer, for each of the six environments. The dashed black line is a linear least-squares fit of the curve averaged over the six environments.

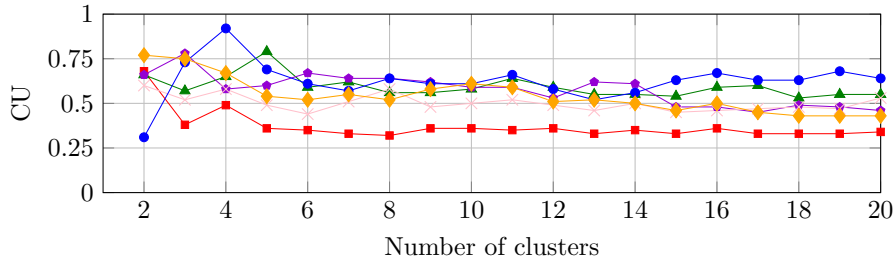
of approximately +0.03 per layer. This suggests that, despite the environment-specific variations, deeper representations tend on average to provide a better trade-off between CU and AS.

1.4.2 Analysis of the selected layers

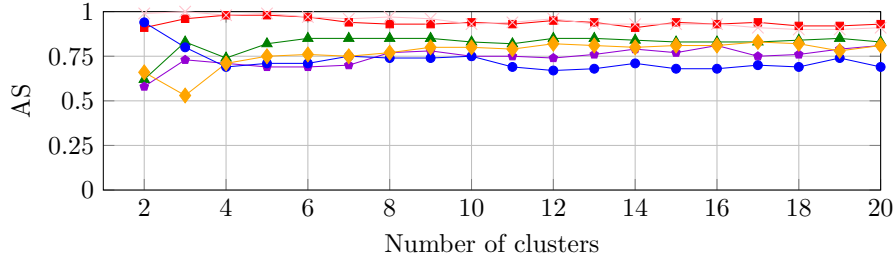
The second analysis focuses on the layer that provides the best trade-off between consistency and abstraction for each environment. For each environment, the layer with the highest average ACU across the tested cluster numbers is selected. Table 1.4 reports the selected layer, together with the corresponding average CU, AS, and ACU.

The selected layer is the last analyzed layer for **Mario Bros**, **Ms. Pac-Man**, and **Surround**, while an intermediate layer is selected for **Breakout**, **Pong**, and **Space Invaders**. The metrics for these selected layers are then examined across the different numbers of clusters. Figure 1.6 reports the evolution of CU, AS, and ACU as the number of clusters varies.

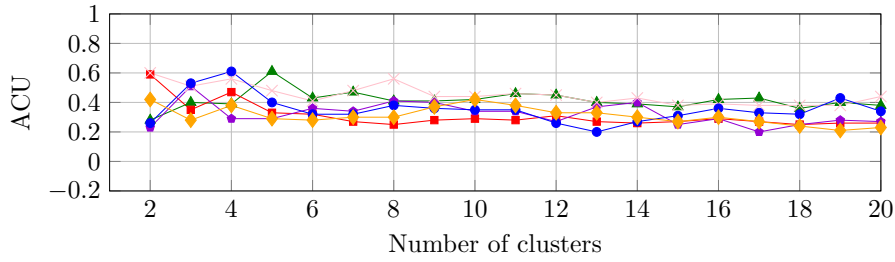
Pong achieves the highest average ACU among the six environments, with a value of 0.45 ± 0.07 . This results from a moderate CU combined with an AS that saturates from the third layer onward. In contrast, **Mario Bros** achieves the highest AS, with 0.94 ± 0.02 , but also the lowest CU, with 0.37 ± 0.08 . The resulting ACU is therefore among the lowest across the six environments.



(a) CU with respect to the number of clusters.



(b) AS with respect to the number of clusters.



(c) ACU with respect to the number of clusters.



Figure 1.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

Environment	Layer	CU	AS	ACU
Breakout	6b	0.59 ± 0.06	0.82 ± 0.05	0.41 ± 0.06
Mario Bros	8b	0.37 ± 0.08	0.94 ± 0.02	0.31 ± 0.08
Ms. Pac-Man	8b	0.58 ± 0.08	0.74 ± 0.05	0.32 ± 0.07
Pong	4	0.50 ± 0.04	0.95 ± 0.03	0.45 ± 0.07
Space Invaders	4	0.63 ± 0.11	0.73 ± 0.06	0.35 ± 0.09
Surround	8b	0.54 ± 0.10	0.77 ± 0.07	0.31 ± 0.06

Table 1.4: Layer that maximizes the average ACU across $K \in [2, 20]$ for each environment, together with the corresponding average CU, AS, and ACU over K .

1.5 Qualitative Analysis

The metrics reported in Section 1.4 evaluate the LS clusterings quantitatively but do not indicate what the resulting clusters actually represent. This section complements them with a qualitative, hand-labeled inspection of the clusters obtained for each environment, in the spirit of Figure ?? . For each environment, k -means is applied to a fixed layer and number of clusters K , chosen for the interpretability of the resulting groupings rather than for the highest ACU, and a sample of observations from each cluster is inspected to propose a semantic label.

1.5.1 Breakout

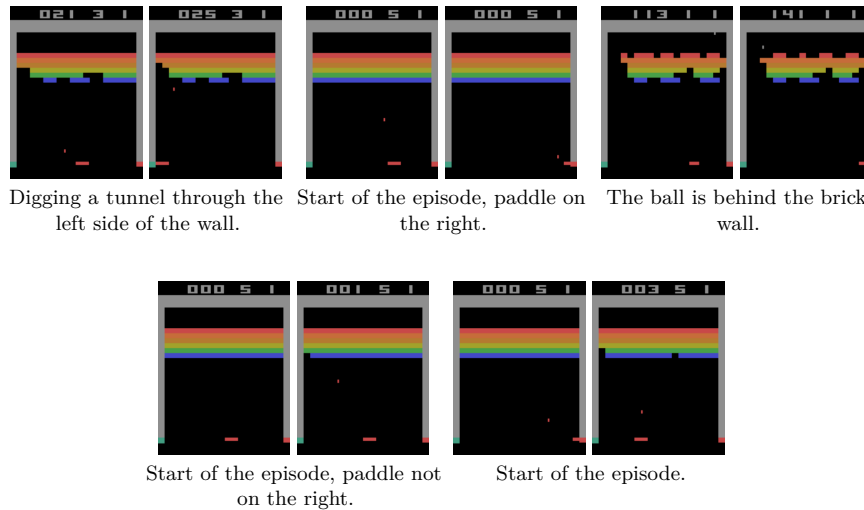
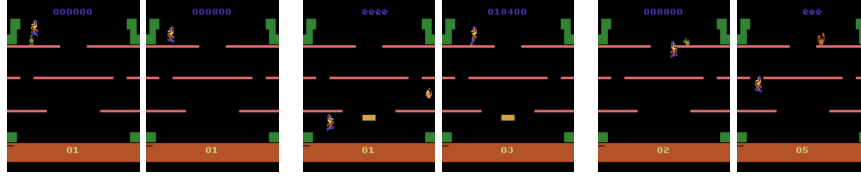


Figure 1.7: Clusters identified by CAR in the **Breakout** environment (layer 6b, $K = 5$).

Figure 1.7 shows the clusters obtained at layer 6b with $K = 5$. The clustering separates the beginning of the episode into three clusters distinguished mainly by the paddle's position, and isolates two more advanced game states, one in which the agent has dug a tunnel through the left side of the wall and one in which the ball is behind the wall.

1.5.2 Mario Bros



Mario fights enemies on the left side of the screen.

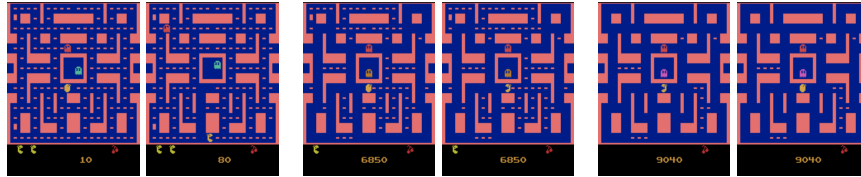
Mario hits the POW block.

Mario fights enemies on the right side of the screen.

Figure 1.8: Clusters identified by CAR in the **Mario Bros** environment (layer 3, $K = 3$).

Figure 1.8 shows the clusters obtained at layer 3 with $K = 3$. Rather than separating phases of the episode, the clustering separates the side of the screen on which Mario is fighting enemies, together with a third cluster corresponding to hitting the POW block, a fixed, recognizable location in the level.

1.5.3 Ms. Pac-Man



Start of the episode.

Middle of the episode.

End of the episode.

Figure 1.9: Clusters identified by CAR in the **Ms. Pac-Man** environment (layer 6b, $K = 3$).

Figure 1.9 shows the clusters obtained at layer 6b with $K = 3$. Unlike **Mario Bros**, the clustering here separates the coarse temporal phase of the episode, tracked through the score and the number of pellets remaining in the maze, rather than a spatial feature of the observation.

278 1.5.4 Pong

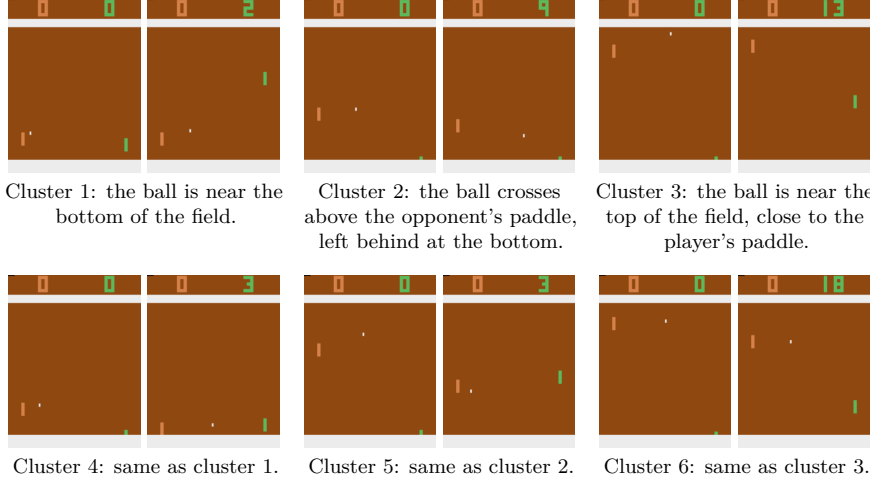
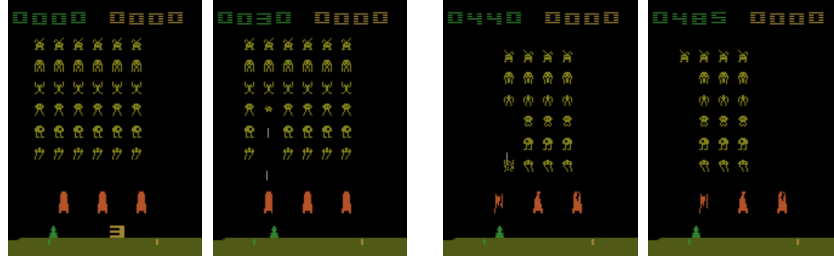


Figure 1.10: Clusters identified by CAR in the **Pong** environment (layer 4, $K = 6$). The bottom row duplicates the semantics of the top row, illustrating over-segmentation at this value of K .

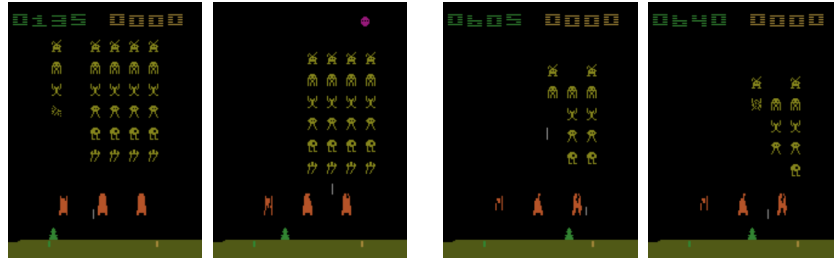
279 Figure 1.10 shows the clusters obtained at layer 4 with $K = 6$. Inspection of the
 280 six clusters reveals only three distinct semantics, each split into two clusters: the
 281 ball near the bottom of the field, the ball crossing the court above the opponent's
 282 paddle left behind at the bottom, and the ball near the top of the field close to
 283 the player's paddle. This is an instance of over-segmentation, in which increasing
 284 K duplicates existing concepts instead of revealing new ones, consistent with
 285 the drop of CU observed at high K for this environment in Figure 1.6.

1.5.5 Space Invaders



Start of the episode.

Middle of the episode.



The left column has been cleared; the flying saucer occasionally appears.

Late in the episode, few invaders remain.

Figure 1.11: Clusters identified by CAR in the **Space Invaders** environment (layer 4, $K = 4$).

Figure 1.11 shows the clusters obtained at layer 4 with $K = 4$. Three clusters track the coarse progression of the episode, from the initial, intact formation of invaders to the sparse, late-game configurations, while the fourth isolates a more specific behavior in which the agent has cleared the leftmost column and is positioned to catch the flying saucer that occasionally crosses the top of the screen.

293 1.5.6 Surround

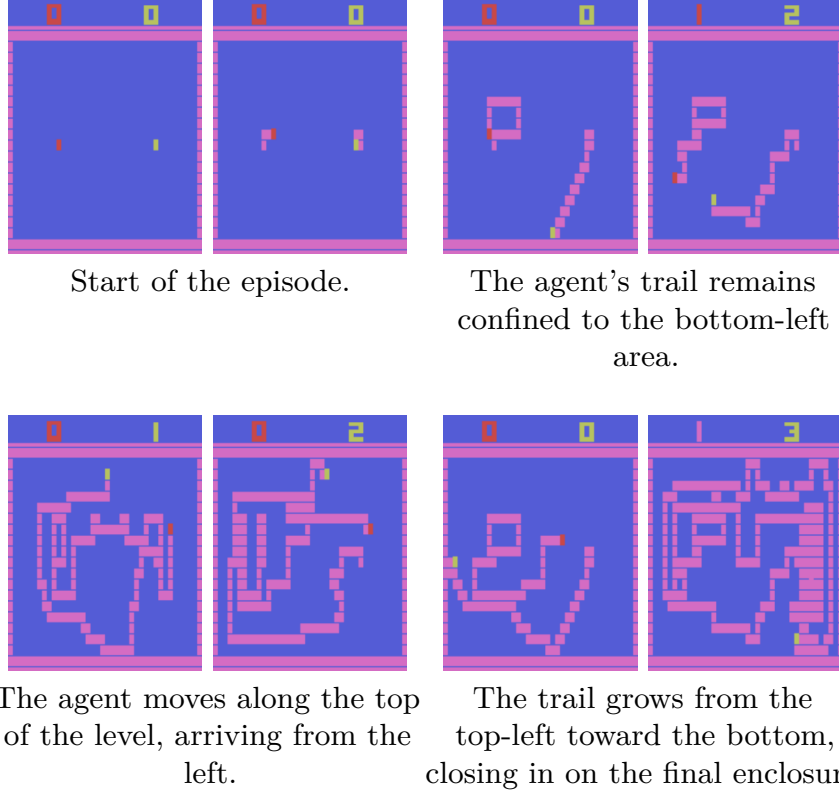


Figure 1.12: Clusters identified by CAR in the **Surround** environment (layer 8b, $K = 4$).

294 Figure 1.12 shows the clusters obtained at layer 8b with $K = 4$. Besides the
 295 start of the episode, the clustering separates configurations by the region of the
 296 level occupied by the agent's trail, from a small trail confined to the bottom-left,
 297 to one running along the top after arriving from the left, to a dense trail that
 298 has grown from the top-left toward the bottom and is closing in on the final
 299 enclosure between the two players.

300 1.6 Conclusion

301 The experiments reported in this chapter apply CAR and its metrics to five Atari
 302 environments. The surrogate policies reproduce the behavior of the initial policies
 303 with an average *Behavior Reconstruction* of 0.94 ± 0.02 . The latent clusterings
 304 are stable across independently trained surrogate policies, with a *Clustering*
 305 *Universality* of about 0.5 from three clusters onward, and they are largely
 306 independent of the observation and action spaces, with an average *Abstraction*
 307 *Score* of 0.99 ± 0.00 . Together, these results provide empirical support for the
 308 *Representation Convergence Hypothesis* (Hyp. ??) and, more broadly, for the
 309 hypothesis-driven approach underlying CAR.

Across all six environments, CU tends to be highest in the earliest layers and to decrease with depth, while AS follows the opposite trend, increasing with depth as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization (Figure 1.5). ACU, which combines both effects, therefore peaks once the representation is abstract enough for AS to be high, but before CU has decayed too much. For **Mario Bros**, **Ms. Pac-Man**, and **Surround**, this optimum is reached at the last analyzed layer, suggesting that further gains might be observed by analyzing even deeper representations. For **Breakout**, **Pong**, and **Space Invaders**, in contrast, ACU peaks at an intermediate layer and decreases again toward the last one, indicating that the deepest representation is not always the most informative for CAR. This confirms that the layer at which the analysis is performed is not a neutral methodological choice and should itself be selected according to ACU rather than fixed a priori to the last convolutional layer, as was done in the single-layer analysis reported in earlier work.

Several other methodological choices have not yet been evaluated. These include the NN architecture, the distance used to compare embeddings, and the clustering algorithm. Their impact should be studied jointly to determine how much the discovered Concept Structure (CS) depends on the analysis procedure rather than on the policy itself.

The evaluation is also limited in scope. It considers only five Atari games, and the semantic labels attached to the clusters in Figure ?? were assigned by hand, which is time-consuming and subjective. Extending the study to more diverse environments and developing human-centered evaluation protocols would strengthen the validity of the resulting explanations.

Acronyms

336	ACU	<i>Adjusted Clustering Universality</i>	1, 9–12, 17
337	APPO	Asynchronous Proximal Policy Optimization	2
338	AS	<i>Abstraction Score</i>	1, 9–12, 17
339	CAR	Clustering Agent Representation	1–7, 12–17
340	CS	Concept Structure	17
341	CU	<i>Clustering Universality</i>	1, 9–12, 14, 17
342	DL	Deep Learning	1
343	DQN	Deep Q-Networks	2
344	GPU	Graphics Processing Unit	3
345	LS	Latent Space	9, 12
346	NN	Neural Network	2, 3, 6, 17
347	PPO	Proximal Policy Optimization	2, 7, 8
348	RL	Reinforcement Learning	1–4

Bibliography

- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article.**
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Hafner et al., 2023] Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web.**
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article.**
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.

- 384 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
385 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
386 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
387 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
388 imperative style, high-performance deep learning library. In Advances in
389 Neural Information Processing Systems (NeurIPS).
- 390 [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine
391 learning. <https://github.com/rapidsai/cuml>.
- 392 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
393 Klimov, O. (2017). Proximal policy optimization algorithms.
- 394 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
395 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
396 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
397 environments.